

Processes and Programs

A program is a passive artifact: instructions and static data sitting in a file. A process is that program in motion, with a program counter, registers, a stack, a heap, and open file descriptors. The operating system tracks each process in a process control block (PCB), which records the process identifier, scheduling state, saved CPU context, and accounting information. Because the PCB captures everything needed to resume execution, the kernel can pause a process and continue it later as if nothing happened. Processes move through a small set of states: new, ready, running, waiting, and terminated. Only one process per core is running at a time; the ready queue holds processes that could run immediately if given a core, while waiting processes are blocked on some event such as disk input or a timer.

Threads and Context Switching

Threads are execution streams that live inside a process and share its address space, open files, and global data, while keeping private stacks and register state. Sharing an address space makes communication between threads cheap, but it also means one careless write can corrupt data another thread is using. Creating a thread is far cheaper than creating a process because no new address space must be built. A context switch is the act of saving the current execution context into its PCB or thread control block and loading another. Switches are pure overhead: the machine spends cycles on bookkeeping and, worse, loses cache and TLB warmth, so the freshly scheduled thread starts by missing in caches it used to own. Schedulers therefore try to balance responsiveness against switching too often.

CPU Scheduling Basics

The scheduler decides which ready process runs next. Common metrics are turnaround time (submission to completion), waiting time (time spent in the ready queue), and response time (submission to first output). First-come-first-served (FCFS) is simple but suffers from the convoy effect: one long job makes every short job behind it wait. Shortest-job-first (SJF) provably minimizes average waiting time, because moving a short job ahead of a long one reduces the waiting time of the short job by more than it increases the waiting time of the long one. The catch is that run times are not known in advance, so real systems estimate them from history. Preemptive schedulers may interrupt a running process when a better candidate arrives; non-preemptive schedulers wait for the process to yield.

Round-Robin and Multilevel Feedback

Round-robin scheduling gives each process a fixed time quantum and cycles through the ready queue. The quantum is a tuning knob: a very large quantum degrades round-robin into FCFS, while a very small quantum makes the system spend most of its time context switching instead of doing useful work. A common rule of thumb is to size the quantum so that most interactive bursts finish within it. The multilevel feedback queue (MLFQ) refines this idea with several queues of different priority. New jobs start in the top queue; jobs that consume their full quantum are demoted, while jobs that block early stay high priority. In effect MLFQ learns each job's behavior: interactive jobs remain responsive, and long CPU-bound jobs sink to lower queues where they run with larger quanta.

Virtual Memory and Paging

Virtual memory gives every process the illusion of a large, private, contiguous address space, decoupling the addresses a program uses from the physical RAM that backs them. Under paging, the virtual address space is divided into fixed-size pages, and physical memory into frames of the same size. A per-process page table maps each virtual page to a physical frame, along with permission bits and a valid bit. This indirection buys three things: processes are protected from each other because they simply cannot name each other's frames; memory need not be contiguous, which ends external fragmentation; and pages can live on disk until they are actually touched, allowing programs larger than physical memory to run.

The TLB and Page Faults

Walking a page table on every memory access would double or triple memory traffic, so processors keep a translation lookaside buffer (TLB), a small cache of recent virtual-to-physical translations. Because programs exhibit locality, a small TLB catches the overwhelming majority of translations, making paging affordable in practice. When a program touches a page whose valid bit is clear, the hardware raises a page fault and traps to the kernel. Under demand paging, the kernel finds the page's contents (on disk, or zero-filled for new allocations), picks a free frame, updates the page table, and restarts the faulting instruction. The process is unaware anything happened except time passed. Faults that reference truly invalid addresses instead terminate the process with a segmentation fault.

Page Replacement

When memory is full, bringing a page in means choosing a victim page to evict. First-in-first-out (FIFO) evicts the oldest resident page, but it can behave badly: Belady's anomaly shows that with FIFO, giving a workload more frames can actually increase the number of page faults.

Least-recently-used (LRU) evicts the page untouched for the longest time, on the theory that the recent past predicts the near future. True LRU is expensive in hardware, so real systems approximate it with the clock algorithm: pages sit on a circular list with a reference bit, and the clock hand clears bits as it sweeps, evicting the first page it finds whose bit is already clear. A page's reference bit thus acts as a one-bit memory of recent use.

Concurrency, Races, and Locks

A race condition occurs when the result of a computation depends on the timing of concurrent threads. The classic example is two threads incrementing a shared counter: each performs load, add, store, and if the operations interleave, one increment is lost. The code region that touches shared state is a critical section, and correctness requires mutual exclusion: at most one thread inside at a time. A mutex (lock) provides this: threads acquire the lock before entering and release it after leaving; a thread that finds the lock held must wait. Locks restore atomicity, meaning the section appears to execute as one indivisible step. The costs are potential waiting, the risk of forgetting a lock, and the risk of deadlock when multiple locks interact.

Deadlock

Deadlock is a cycle of threads each waiting for a resource the next one holds, so none can proceed. Four conditions must all hold: mutual exclusion (resources cannot be shared), hold and wait (a thread keeps what it has while waiting for more), no preemption (resources cannot be forcibly taken), and circular wait (a cycle of waiters exists). Break any one condition and deadlock is impossible. The most practical strategy is to break circular wait by imposing a global lock ordering: if every thread acquires locks in the same agreed order, a cycle of waiters cannot form. Deadlock avoidance schemes such as the banker's algorithm instead keep the system in states from which all threads can still finish, but they require knowing maximum demands in advance, which is rare.

File Systems and Inodes

A file system maps human-friendly names onto blocks of storage. In Unix-style designs, a file's metadata lives in an inode: size, permissions, timestamps, and pointers to the data blocks. Crucially, the inode does not contain the file's name. Directories are themselves small files holding name-to-inode-number entries, which is why a single inode can be reached by several names (hard links) and why renaming a file does not move its data. Small files are reached directly through the inode's direct pointers, while large files use indirect blocks: pointers to blocks of pointers. Journaling file systems write an intent record to a log before updating metadata in place, so that after a crash, recovery replays or discards whole operations instead of leaving the tree half-updated.